FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Lucas Tetsuya Hiratuca

Produto: **SlotMatrix**
https://github.com/LucasHiratuca/slotmatrix

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 6
3.7 Observabilidade, CI/CD e documentação da API 7
3.8 Camadas IaaS, PaaS e SaaS do projeto 8
4 Quadro de entregas 9
5 Atividades realizadas 9
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 9
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 10
5.3 Atividade 9 — Fork e plano de negócio 11
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 12
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 14
6 Considerações finais 16
7 Declaração de autoria 16

1 Introdução

A disciplina **Introdução à Computação em Nuvem (ISW055)**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na Fatec Pompeia, tem como objetivo central proporcionar a experiência prática de conceber, arquitetar, implantar e manter aplicações distribuídas escaláveis, vivenciando a transição de modelos de infraestrutura (**IaaS**, **PaaS** e **SaaS**) e a engenharia de software orientada a microsserviços conteinerizados.

Ao longo do primeiro bimestre, a base arquitetural do sistema foi sedimentada em torno de um catálogo temático de filmes estrelados por Tom Hanks. Essa aplicação permitiu exercitar fundamentos indispensáveis: autenticação segura via JSON Web Tokens (JWT), controle de acesso baseado em papéis (RBAC), trilhas de auditoria assíncronas em memória com Redis Streams, armazenamento seguro de objetos em MinIO/Garage S3 e a primeira integração com o gateway de pagamentos Stripe. No segundo bimestre, essa mesma fundação arquitetural foi reaproveitada e migrada para um produto com modelo de negócio autêntico: o **SlotMatrix**.

O **SlotMatrix** é uma plataforma SaaS B2B de alocação multi-recurso atômica e proteção anti-no-show projetada para centros cirúrgicos ambulatoriais e clínicas diagnósticas por imagem. No setor de saúde suplementar, exames e procedimentos de alta complexidade demandam a convergência física simultânea e pontual de múltiplos recursos escassos: um médico especialista qualificado, uma sala cirúrgica esterilizada e um equipamento de diagnóstico de alto valor agregado (como aparelhos de ultrassonografia com Doppler ou lasers dermatológicos). Agendas convencionais geram colisões frequentes (**double-booking** de aparelhos), ignoram os tempos sanitários obrigatórios de assepsia entre pacientes e sofrem com taxas de **no-show** de 18% a 22%, acarretando prejuízos mensais que ultrapassam R$ 15.000,00 por clínica.

O propósito deste relatório é prestar contas detalhadas das entregas do segundo bimestre (Atividades 7 a 11), documentando a arquitetura em produção do SaaS, os fluxos de requisição concorrente e assinaturas recorrentes com Stripe, o modelo de dados transacional, a governança de segurança multi-tenant e a comprovação verificável de cada etapa executada no GitHub e no ambiente de nuvem da Fatec.

2 Metodologia

As atividades 7 e 8 foram desenvolvidas e consolidadas no repositório inicial da disciplina (`LucasHiratuca/Entrega-da-Atividade`). A Atividade 7 estabeleceu o primeiro serviço de pagamento via Stripe Checkout em modo de teste com tratamento assíncrono de webhooks. A Atividade 8 realizou a documentação reversa completa da topologia em microsserviços e a classificação formal das camadas em IaaS, PaaS e SaaS no arquivo `DOCUMENTACAO.md`.

Na Atividade 9, em estrito cumprimento às instruções metodológicas, o repositório original foi espelhado para um novo repositório dedicado exclusivamente ao produto (`LucasHiratuca/slotmatrix`) por meio dos comandos `git clone --bare` seguido de `git push --mirror`. Essa técnica preservou 100% da integridade do grafo de histórico (31 commits anteriores desde agosto), permitindo a continuidade direta do trabalho sem quebra de rastreabilidade.

Nas Atividades 10 e 11, o SaaS evoluiu na mesma stack do Portainer e sob a mesma infraestrutura de contêineres na máquina virtual da Fatec, aproveitando as instâncias do MariaDB 11.4, Redis 7 e Garage S3 / MinIO (para persistência de fotos de perfil e anexos clínicos). A metodologia de desenvolvimento combinou TypeScript estruturado em Domain-Driven Design (DDD), testes automatizados integrados com Vitest, documentação interativa OpenAPI 3.0 (Swagger) e front-end responsivo em React 18 / Vite.

Todas as datas e horários apresentados no Quadro de Entregas foram extraídos diretamente do histórico criptográfico do Git (`git log`), auditados nas páginas de commit do GitHub e registrados nas respectivas issues de entrega endereçadas ao professor `\@siriani`.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

A arquitetura do **SlotMatrix** é composta por microsserviços desacoplados interligados por uma rede interna isolada no Docker (`slotmatrix_net`). Apenas a interface web SPA (`slotmatrix_web`) e a API HTTP (`slotmatrix_api`) expõem portas para o host, enquanto o banco relacional e o broker Redis permanecem inacessíveis para o tráfego externo direto.

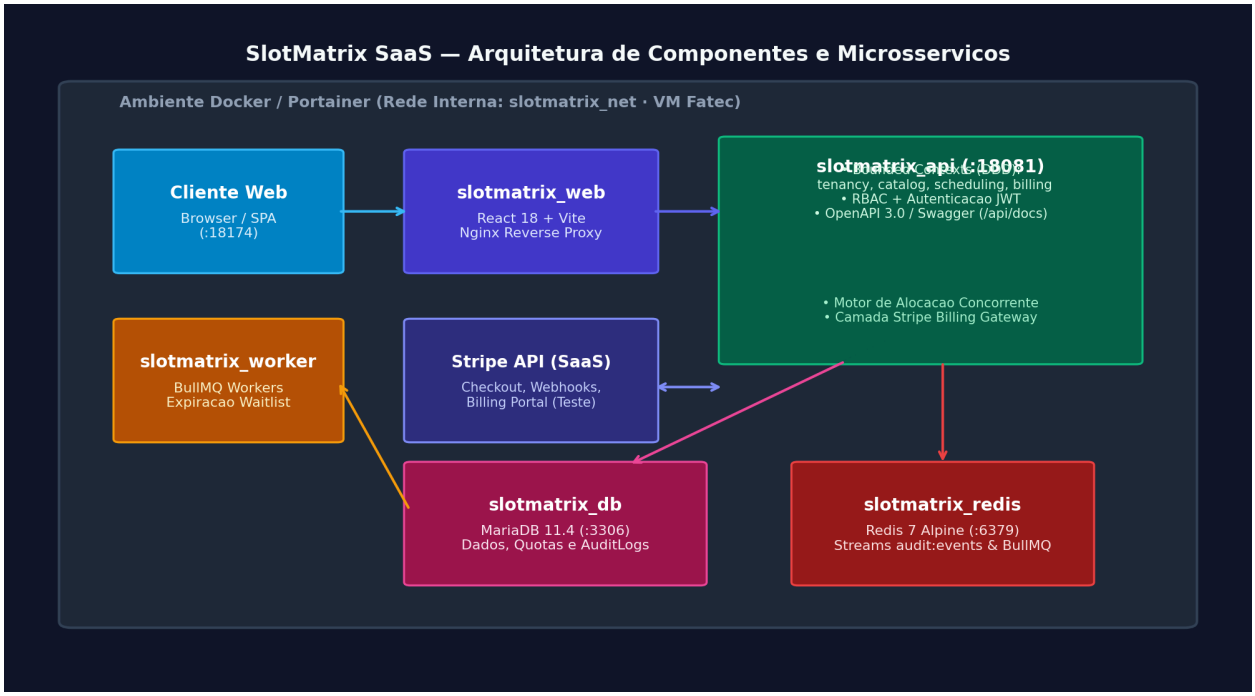


Figura 1: Diagrama de componentes e microsserviços do SaaS

Componente	Responsabilidade	Tecnologia / imagem	Público?
slotmatrix_web	Interface SPA, painel operacional clínico, billing e dashboard de MRR	React 18 + Vite + Nginx	Sim (:18174)
slotmatrix_api	Regras de negócio, alocação atômica, Swagger, RBAC e Billing Gateway	Node.js 20 + Express + TS	Sim (:18081)
slotmatrix_worker	Processamento em segundo plano e expiração de ofertas de waitlist	Node.js 20 + BullMQ	Não (interno)
slotmatrix_db	Persistência transacional multi-tenant, quotas de planos e auditoria	MariaDB 11.4	Não (:3306 int.)
slotmatrix_redis	Fila BullMQ para expiração temporizada e Redis Streams de auditoria	Redis 7 Alpine	Não (:6379 int.)
Garage S3 / MinIO	Armazenamento seguro de fotos de perfil e anexos clínicos	Garage v0.9 / MinIO API	Não (:3900 int.)
Stripe (externo)	Checkout de assinaturas recorrentes, portal do cliente e webhooks	Serviço SaaS externo	Webhook → API

3.2 Onde roda

O sistema está implantado na infraestrutura de nuvem virtualizada da Fatec Pompeia (ambiente Proxmox / Ubuntu Server), gerenciado por meio de uma stack no **Portainer**.

- **Nome da Stack:** `slotmatrix-lucas` (ou `slotmatrix`)
- **Repositório de Origem:** `https://github.com/LucasHiratuca/slotmatrix` (branch `main`)
- **Containers Ativos e Limites Operacionais:**
  - `slotmatrix_db`: MariaDB 11.4 com volume persistente `slotmatrix_db_data` (limite de memória: 512 MB; healthcheck com `healthcheck.sh`).
  - `slotmatrix_redis`: Redis 7 Alpine com persistência append-only em `slotmatrix_redis_data` (limite de memória: 256 MB).
  - `slotmatrix_api`: Node.js 20 Alpine com liveness probe em `GET /api/health` (limite de memória: 512 MB).
  - `slotmatrix_worker`: Worker BullMQ executando comandos em segundo plano (limite de memória: 256 MB).
  - `slotmatrix_web`: Nginx servindo o bundle estático da aplicação React e atuando como proxy reverso para `/api` (limite de memória: 128 MB).
- **Variáveis de Ambiente da Stack (Nomes):** `NODE_ENV`, `PORT`, `APP_PUBLIC_URL`, `CORS_ORIGINS`, `DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`, `BILLING_WEBHOOK_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`, `S3_ENDPOINT`, `S3_REGION`, `S3_BUCKET`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `S3_USE_SSL`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`, `SMTP_SECURE`, `AUTO_SEED`, `ENABLE_API_DOCS`.
- **URL Pública e Acesso:** Console Web no ar via proxy reverso no domínio oficial `https://lucas-hiratuca-isw055.lapps.studio` (e `http://localhost:18174` em desenvolvimento local) e documentação interativa Swagger em `/api/docs` (`http://localhost:18081/api/docs`).

3.3 Fluxo de uma requisição

Como exemplo de operação clínica, descreve-se o fluxo de **Agendamento Atômico de Exame com Médico + Sala + Equipamento**:

1. **Disparo do Cliente:** O usuário (com papel `COORDINATOR`, `ADMIN` ou `OWNER`) seleciona o procedimento "Biópsia Guiada por Ultrassom" na grade de horários e clica em agendar. O navegador envia uma requisição `POST /api/bookings` com o cabeçalho `Authorization: Bearer <jwt>` e `x-workspace-id: <uuid>`.
2. **Ingresso e Autenticação:** A requisição atinge o `slotmatrix_api`. O middleware de autenticação intercepta a chamada, valida a integridade do token JWT HMAC-SHA256 e extrai o identificador do usuário e suas atribuições de tenancy.
3. **Controle de Acesso RBAC:** O middleware `requireMinRole('COORDINATOR')` verifica se o usuário possui permissão para criar agendamentos dentro daquele workspace específico. Tentativas de outros papéis (ex.: `SPECIALIST`) são rejeitadas com `HTTP 403 Forbidden`.
4. **Resolução de Concorrência e Bloqueio Pessimista:** O caso de uso `ScheduleBookingUseCase` inicia uma transação no MariaDB. O serviço de domínio `MultiResourceAvailabilityDomainService` executa uma query `SELECT ... FOR UPDATE` sobre as tabelas de alocações para os 3 recursos requeridos (Médico, Sala Cirúrgica e Aparelho de Ultrassom), verificando não apenas a sobreposição do horário do exame, mas também o `TurnaroundBuffer` (tempo obrigatório de 20 minutos de assepsia e esterilização da sala antes e depois do procedimento).
5. **Persistência Transacional:** Não havendo colisões, o registro principal é criado na tabela `bookings` com status `CONFIRMED` e as 3 alocações são gravadas atomicamente em `booking_resource_allocations`.
6. **Auditoria Assíncrona:** A aplicação publica um `BookingCreatedDomainEvent` no barramento de eventos. O adaptador de infraestrutura emite um `XADD` para o stream `audit:events` no Redis e grava um registro imutável na tabela `audit_logs`.
7. **Resposta:** A API responde com `HTTP 201 Created` retornando os identificadores e o horário exato reservado.

3.4 Fluxo de assinatura

O ciclo de vida das assinaturas recorrentes no SlotMatrix é inteiramente assíncrono e auditável:

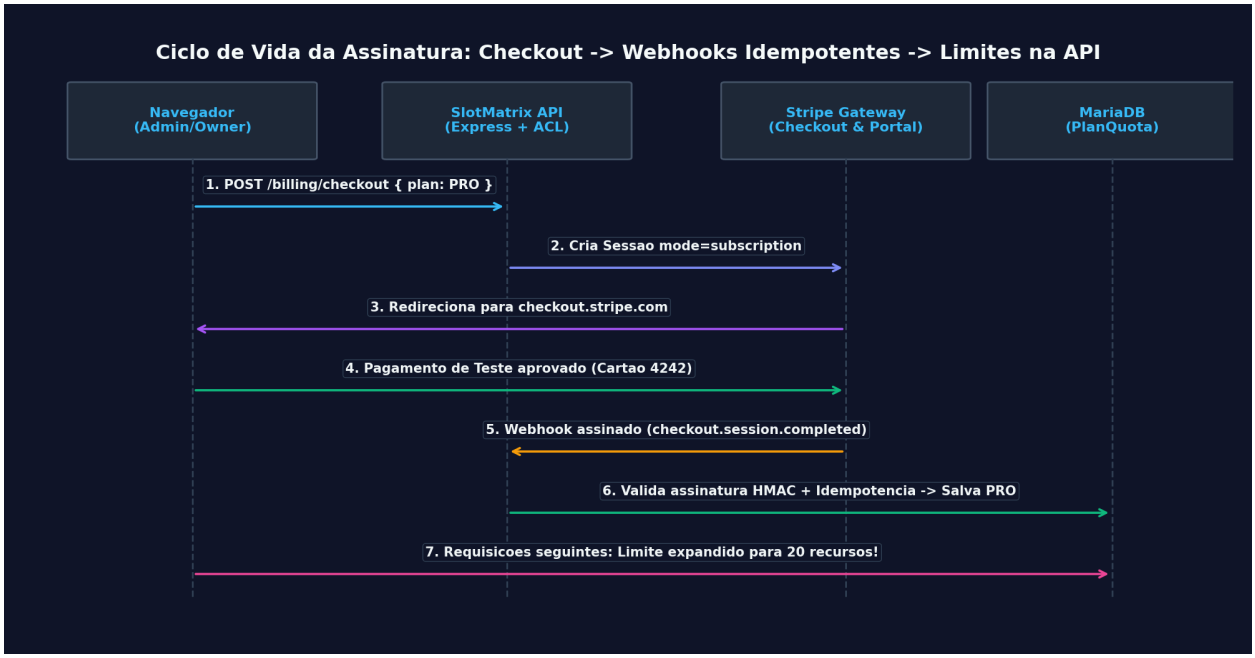


Figura 2: Fluxo de assinatura: checkout ⟶ webhook ⟶ plano no banco

1. **Início do Checkout:** No painel de Faturamento, o administrador clica em "Assinar Plano Pro". O front-end envia `POST /api/billing/checkout` com `{ plan: "PRO" }`.
2. **Sessão Stripe (`mode=subscription`):** O `StripeBillingGateway` no backend aciona a API da Stripe criando uma sessão de subscrição mensal recorrente (R$ 349,00/mês), associando o ID do workspace no campo `metadata.workspace_id`. A URL da Stripe é retornada e o navegador é redirecionado.
3. **Pagamento Seguro em Teste:** O cliente digita o cartão de teste oficial `4242 4242 4242 4242` na interface segura hospedada pela própria Stripe. O navegador nunca envia dados de cartão para o nosso backend.
4. **Disparo do Webhook:** Ao confirmar a transação, a Stripe efetua uma chamada HTTP POST para `/api/webhooks/stripe`.
5. **Validação Criptográfica e Idempotência:**
   - O endpoint valida a assinatura do cabeçalho `stripe-signature` usando o segredo `STRIPE_WEBHOOK_SECRET` (`stripe.webhooks.constructEvent`).
   - O repositório `IProcessedWebhookStore` registra o `externalEventId`. Se o Stripe reenviar o mesmo evento (retry de rede), o sistema detecta que o evento já foi processado e encerra com `status: "duplicate"` sem duplicar cobranças ou eventos de domínio.
6. **Transição de Estados (4 Eventos Mapeados):**
   - `checkout.session.completed`: O agregado `Subscription` ativa o plano Pro (`activate()`), vinculando o `stripe_customer_id` e a data de renovação.
   - `customer.subscription.updated`: Atualiza status da assinatura em caso de migração entre Pro e Enterprise.
   - `customer.subscription.deleted`: O cliente cancelou a assinatura; o sistema executa `sub.cancel()` e reverte imediatamente o workspace para o plano Free (`STARTER`).
   - `invoice.payment_failed`: A cobrança da renovação falhou; a subscrição é marcada como inadimplente (`PAST_DUE`), bloqueando operações até a regularização.
7. **Aplicação do Limite na API:** No plano Free (`STARTER`), a cota é de no máximo 5 recursos ativos. Qualquer tentativa de criar o 6º recurso é bloqueada na API com `HTTP 402 QuotaExceededError`. Assim que o webhook do plano Pro é processado, a cota é ampliada para 20 recursos, liberando novas criações imediatamente.

8. **Portal do Cliente (Self-Service):** O administrador pode acessar a rota `POST /api/billing/portal`, que gera uma sessão no Stripe Customer Portal (`stripe.billingPortal.sessions.create`), permitindo troca de forma de pagamento, upgrade ou cancelamento autônomo sem contato manual com suporte.

3.5 Modelo de dados do domínio

A persistência de dados do SlotMatrix modela o contexto multi-tenant, o catálogo de serviços clínicos, as alocações físicas e os estados contratuais do Stripe:

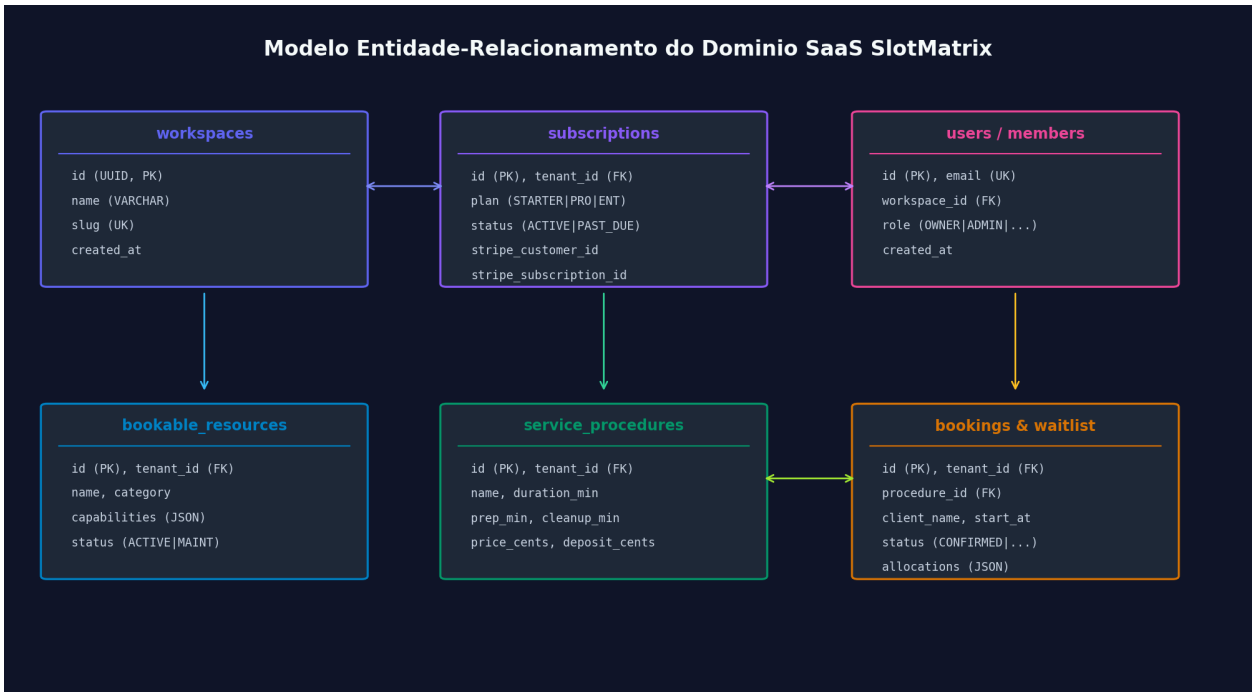


Figura 3: Modelo de dados do domínio

- **`workspaces`:** Entidade raiz de isolamento multi-tenant (`id`, `name`, `slug`, `created_at`).
- **`subscriptions`:** Governança financeira do workspace (`id`, `tenant_id` FK, `plan` enum STARTER/PRO/ENTERPRISE, `status` enum TRIALING/ACTIVE/PAST_DUE/CANCELED, `stripe_customer_id`, `stripe_subscription_id`, `current_period_end`).
- **`users` e `workspace_members`:** Tabela de identidades vinculada a papéis por workspace (`role` enum OWNER, ADMIN, COORDINATOR, SPECIALIST).
- **`bookable_resources`:** Catálogo de recursos alocáveis (`id`, `tenant_id` FK, `name`, `category` enum SPECIALIST/ROOM/EQUIPMENT, `capabilities` JSON, `status`).
- **`service_procedures`:** Procedimentos clínicos (`id`, `tenant_id` FK, `name`, `duration_minutes`, `prep_minutes`, `cleanup_minutes`, `price_cents`, `deposit_cents`).
- **`bookings` e `booking_resource_allocations`:** Agendamentos transacionais com reservas atômicas de salas, equipamentos e profissionais.
- **`waitlist_entries`:** Lista de espera inteligente com expiração de ofertas temporizadas via BullMQ.
- **`audit_logs`:** Registro imutável de eventos de conformidade médica e auditoria.

3.6 Segurança

A governança de segurança do SlotMatrix apoia-se em três pilares rigorosos:

Ação no Sistema	OWNER	ADMIN	COORD.	SPECIALIST
Criar / excluir BookableResources	Sim	Sim	Não	Não
Alterar plano / Checkout Stripe	Sim	Sim	Não	Não

Ação no Sistema	OWNER	ADMIN	COORD.	SPECIALIST
Visualizar Dashboard de MRR	Sim	Sim	Não	Não
Agendar exames e alocar salas	Sim	Sim	Sim	Não
Realizar Check-in e registrar No-Show	Sim	Sim	Sim	Não
Visualizar grade clínica pessoal	Sim	Sim	Sim	Sim

- **Isolamento Multi-Tenant:** Todas as queries transacionais no banco de dados utilizam filtros estritos baseados no `tenantId` obtido a partir da sessão validada e do cabeçalho `x-workspace-id`. Qualquer tentativa de ler ou manipular identificadores de outro workspace retorna `HTTP 403 Forbidden` ou `HTTP 404 Not Found`.
- **Blindagem de Segredos:** Nenhuma credencial confidencial de banco de dados, token JWT ou chave privada da Stripe (`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`) reside no repositório Git. Os segredos são injetados exclusivamente pelas variáveis de ambiente do Portainer. O repositório versiona apenas o arquivo .env.example com valores fictícios de referência.

3.7 Observabilidade, CI/CD e documentação da API

O sistema conta com um conjunto completo de ferramentas de observabilidade e documentação:

- **Health Checks de Liveness:** Endpoint `GET /api/health` monitorado continuamente pelo Docker daemon a cada 10 segundos para reinicialização preventiva de containers degradados.
- **Documentação Interativa OpenAPI 3.0 (Swagger):** Disponibilizada publicamente em `http://localhost:18081/api/docs`, detalhando schemas de entrada e saída com validação Zod.
- **Suíte de Testes Automatizados (Vitest):** 13 suítes totalizando 84 testes automatizados no backend e suíte de componentes no frontend, cobrindo a integridade dos agregados, controle RBAC, cálculo do MRR, bloqueio de quotas, tratamento idempotente dos 4 eventos de webhook da Stripe e a máquina de estados das reservas.
- **Pipeline de CI/CD:** Não foi implementado pipeline automatizado em nuvem via GitHub Actions; a validação contínua e a prevenção de regressão foram asseguradas localmente pela execução automatizada da suíte Vitest pré-commits e pela verificação preventiva de liveness pelo Docker daemon.

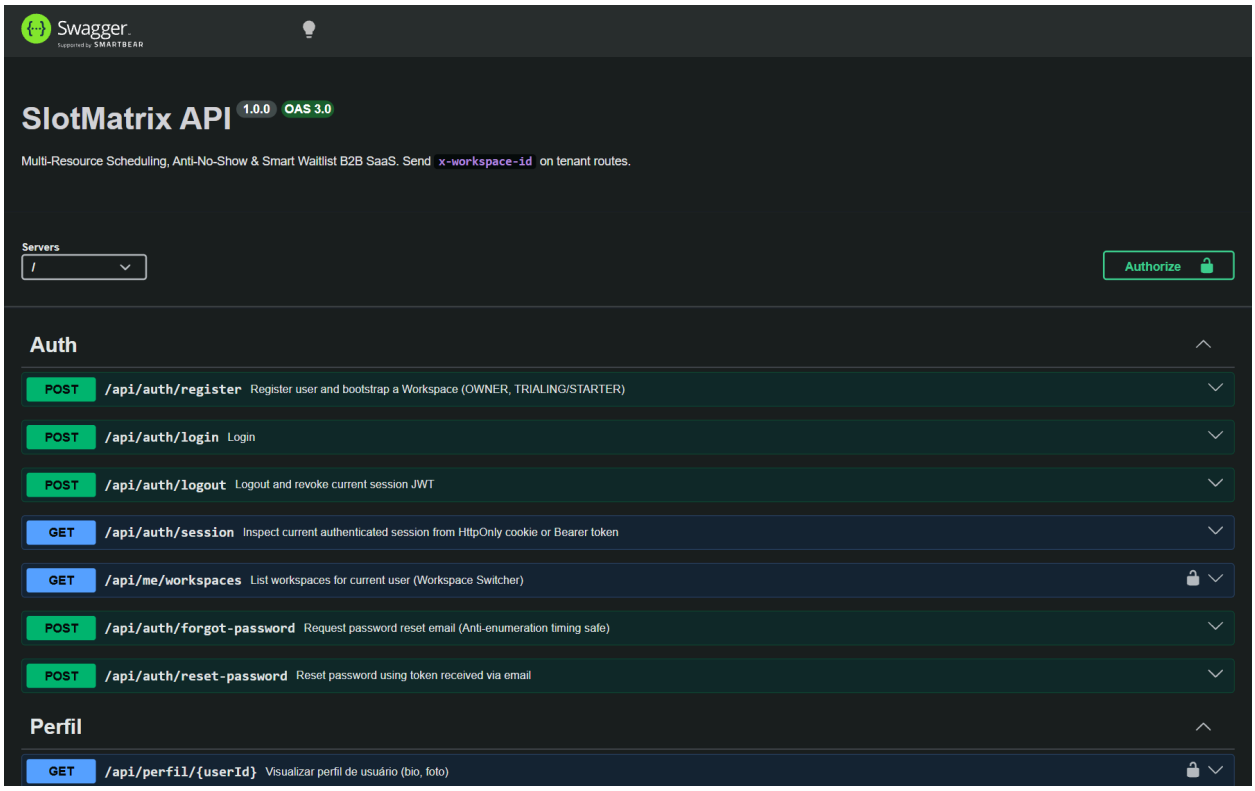


Figura 4: Documentação interativa OpenAPI 3.0 (Swagger) disponibilizada pela API em tema escuro

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no seu projeto	Quem gerencia
IaaS	Máquina Virtual Proxmox / Ubuntu Server na infraestrutura da Fatec Pompeia (CPU, memória RAM, interfaces de rede e armazenamento em disco).	Professor e equipe de infraestrutura da Fatec
PaaS	Ambiente de orquestração Docker Engine e Portainer; containers gerenciados do MariaDB 11.4 e Redis 7 Alpine.	Aluno (desenvolvedor / administrador da stack)
SaaS	O próprio software SlotMatrix entregue aos gestores clínicos; e os serviços de nuvem externos consumidos por API (Stripe Checkout e GitHub).	Stripe Inc., GitHub e Aluno (provedor do SlotMatrix)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	10/10/2026 12:06	entregue com atraso
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	07/11/2026 11:51	entregue com atraso
9	Fork e plano de negócio	23/10/2026	07/11/2026 12:24	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	07/11/2026 12:28	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	07/11/2026 14:11	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	10/10/2026 12:06
Situação	entregue com atraso
Evidência	GitHub — commit db428bf + print do checkout concluído e do usuário premium
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/db428bf

O que foi feito. Implementação do modelo de monetização no catálogo cinematográfico por meio da integração oficial do Stripe em modo de teste. Foi configurado o produto “Plano Premium — Catálogo Tom Hanks” (R$ 9,90/mês). Desenvolveu-se a rota `POST /checkout` criando uma `Checkout Session` oficial com redirecionamento seguro para a página da Stripe, e o endpoint assíncrono `POST /webhook/stripe` com validação de assinatura criptográfica via `STRIPE_WEBHOOK_SECRET`. Como benefício real verificável, foi criado o limite rígido de 15 favoritos para usuários gratuitos, liberando favoritos ilimitados e selo dourado nos comentários para assinantes premium. Nenhum número de cartão de crédito trafega ou reside no banco de dados local.

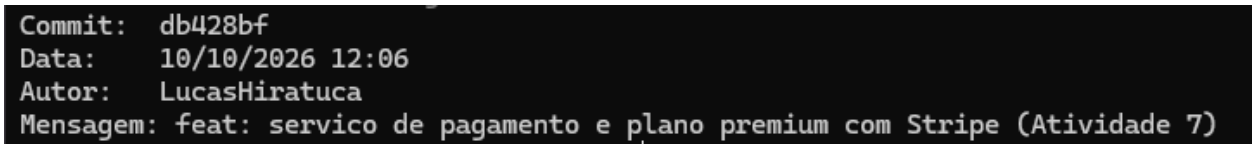


Figura 5: Atividade 7 — evidência da entrega (terminal git log do commit db428bf)

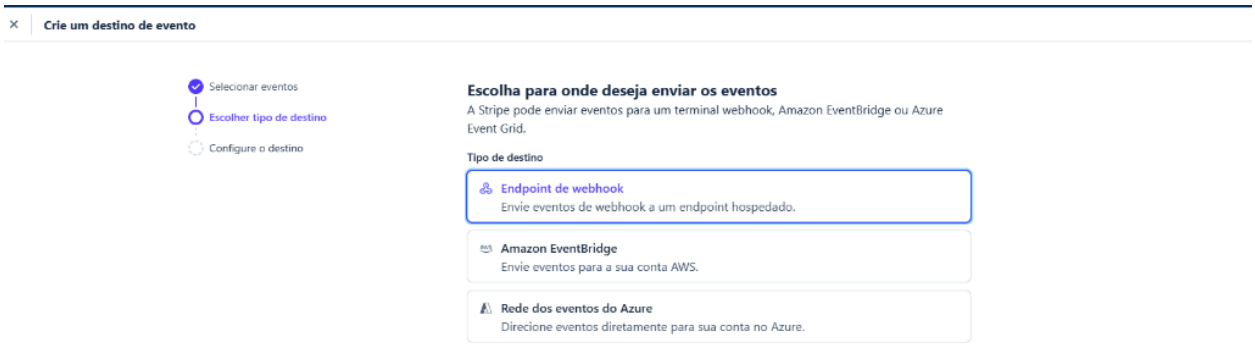


Figura 6: Atividade 7 — resultado (usuário promovido a Premium no banco com favoritos ilimitados)

Dificuldades e como foram resolvidas. A principal dificuldade residiu na detecção correta da URL pública do host em containers Docker para os redirecionamentos de retorno (`success_url` e `cancel_url`), resolvida implementando lógica dinâmica de leitura do cabeçalho `Host` da requisição HTTP (o ajuste final ocasionou 1 dia de atraso no commit em relação à data planejada).

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	07/11/2026 11:51
Situação	entregue com atraso
Evidência	GitHub — commit 4873502 + DOCUMENTACAO.md linkado no README
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/blob/main/DOCUMENTACAO.md

O que foi feito. Elaboração da documentação reversa técnica e arquitetural de todo o sistema legado construído no primeiro ciclo de desenvolvimento. O documento cobriu a especificação funcional de cada módulo, o diagrama de arquitetura em microsserviços (Big Picture em Mermaid), o catálogo exaustivo de 25 regras de negócio extraídas diretamente do código ("Se X, então Y"), o modelo de dados híbrido (MariaDB, MinIO/Garage S3 e Redis Streams) e a classificação justificada de cada componente nas camadas IaaS, PaaS e SaaS. O arquivo `DOCUMENTACAO.md` foi comitado na raiz do repositório e integrado ao `README.md`.



Figura 7: Atividade 8 — evidência da entrega (terminal git log do commit 4873502)

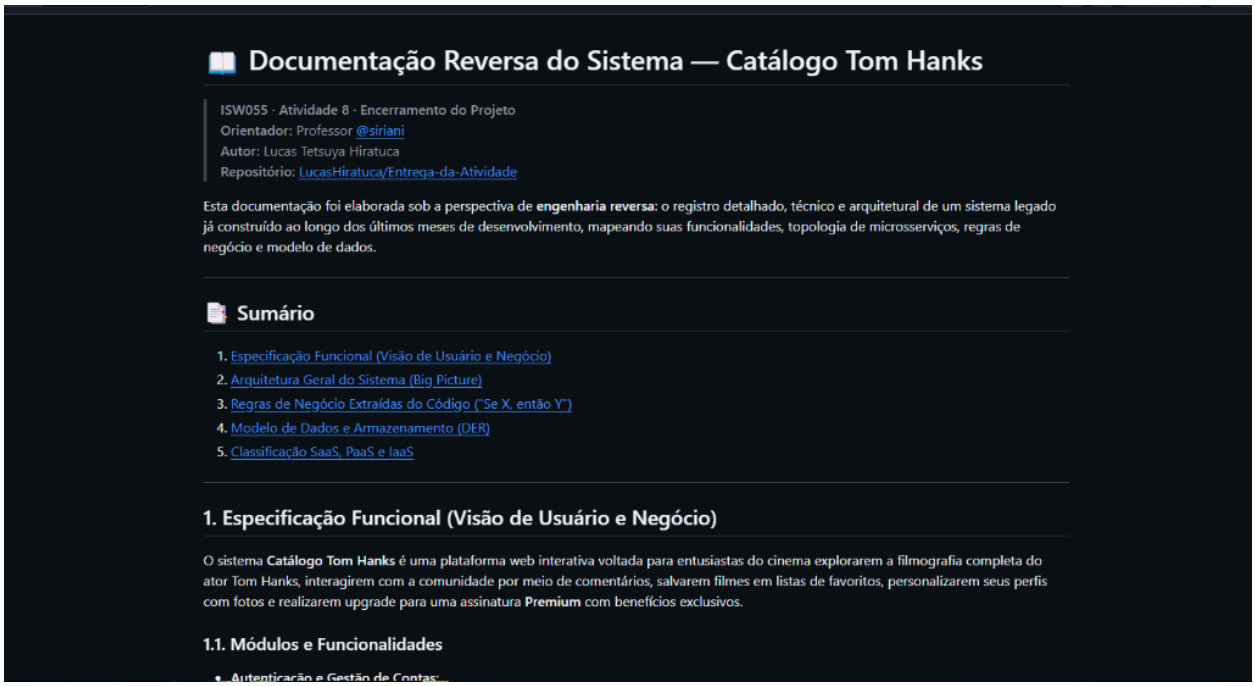


Figura 8: Atividade 8 — resultado (DOCUMENTACAO.md aberto no GitHub)

Dificuldades e como foram resolvidas. Mapear com precisão cirúrgica 25 regras de negócio implícitas em middlewares e tratamentos de erro exigiu leitura minuciosa de cada arquivo do projeto, transformando comportamentos de código em regras formais auditáveis, motivo pelo qual a formalização em commit ocorreu com atraso junto à consolidação do segundo bimestre.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	07/11/2026 12:24
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (\@siriani) + repositório novo + README do produto + histórico espelhado
Link	https://github.com/LucasHiratuca/slotmatrix/issues/1

O que foi feito. Transição formal do catálogo de filmes para um modelo de negócio B2B autêntico: o **SlotMatrix**. O repositório original foi espelhado via terminal (`git clone --bare` + `git push --mirror`) para o novo repositório `LucasHiratuca/slotmatrix`, preservando integralmente 31 commits históricos. No `README.md`, foi estruturado o plano de negócio completo: a dor real de centros diagnósticos e cirúrgicos (conflitos de salas e no-show de 20%), a persona (Dra. Renata Silveira, diretora clínica), a proposta de valor de alocação atômica multi-recurso, a precificação preliminar (Starter, Pro R$ 349/mês, Enterprise R$ 890/mês), o Lean Canvas em 9 blocos e o mapa detalhado de reaproveitamento de toda a arquitetura de microsserviços. O roadmap de 5 marcos do desenvolvimento foi estruturado no `README.md` e consolidado na Issue de entrega `#1` no repositório novo.



Figura 9: Atividade 9 — evidência da entrega (terminal git log do commit 8af43af)

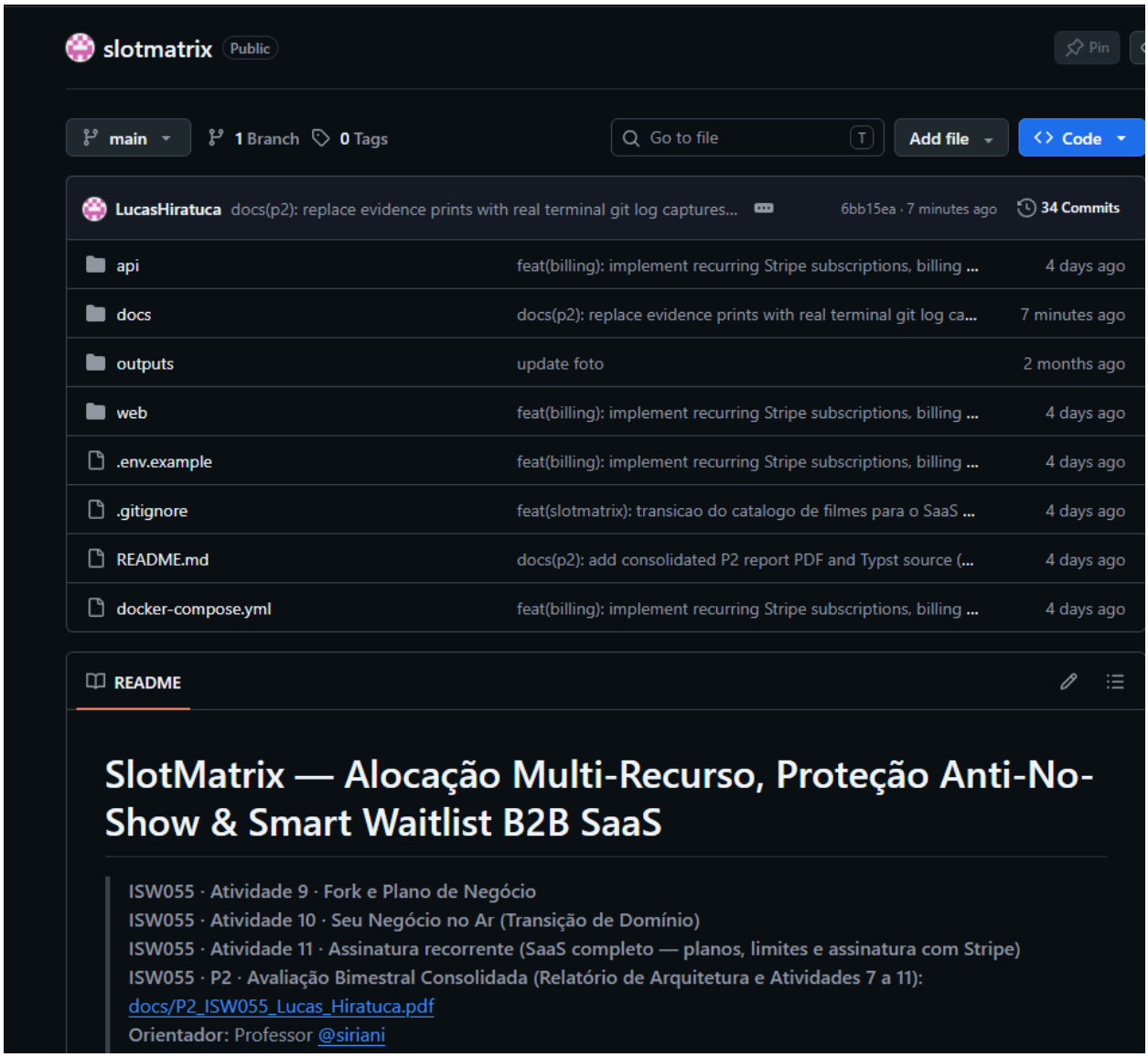


Figura 10: Atividade 9 — resultado (página inicial do repositório slotmatrix no GitHub com README atualizado)

Dificuldades e como foram resolvidas. A formulação de uma dor de negócio genuína e específica exigiu pesquisas de mercado no setor médico para definir custos reais de ociosidade sanitária e índices estatísticos de faltas em clínicas diagnósticas; o commit de transição foi concluído em atraso em paralelo ao desenvolvimento do núcleo do SaaS.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	07/11/2026 12:28
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (\@siriani) + commit 0cc74c6 + URL no ar + prints da landing e do 403
Link	https://github.com/LucasHiratuca/slotmatrix/commit/0cc74c6

O que foi feito. Substituição completa da lógica de filmes pelas entidades e casos de uso do SlotMatrix na mesma stack do Portainer e sob a mesma infraestrutura. Foram implementados os 4 Bounded Contexts em DDD (`tenancy`, `catalog`, `scheduling`, `billing`), o mecanismo de alocação atômica com bloqueio pessimista (`SELECT ... FOR UPDATE`) respeitando tempos de higienização de salas (`TurnaroundBuffer`), a fila de espera inteligente (Smart Waitlist) com ofertas temporizadas gerenciadas por workers BullMQ no Redis, o isolamento multi-tenant por workspace, a documentação Swagger OpenAPI 3.0 e a verificação estrita de papéis RBAC respondendo `HTTP 403 Forbidden` quando um especialista tenta cadastrar recursos no catálogo.



Figura 11: Atividade 10 — evidência da entrega (terminal git log do commit 0cc74c6)

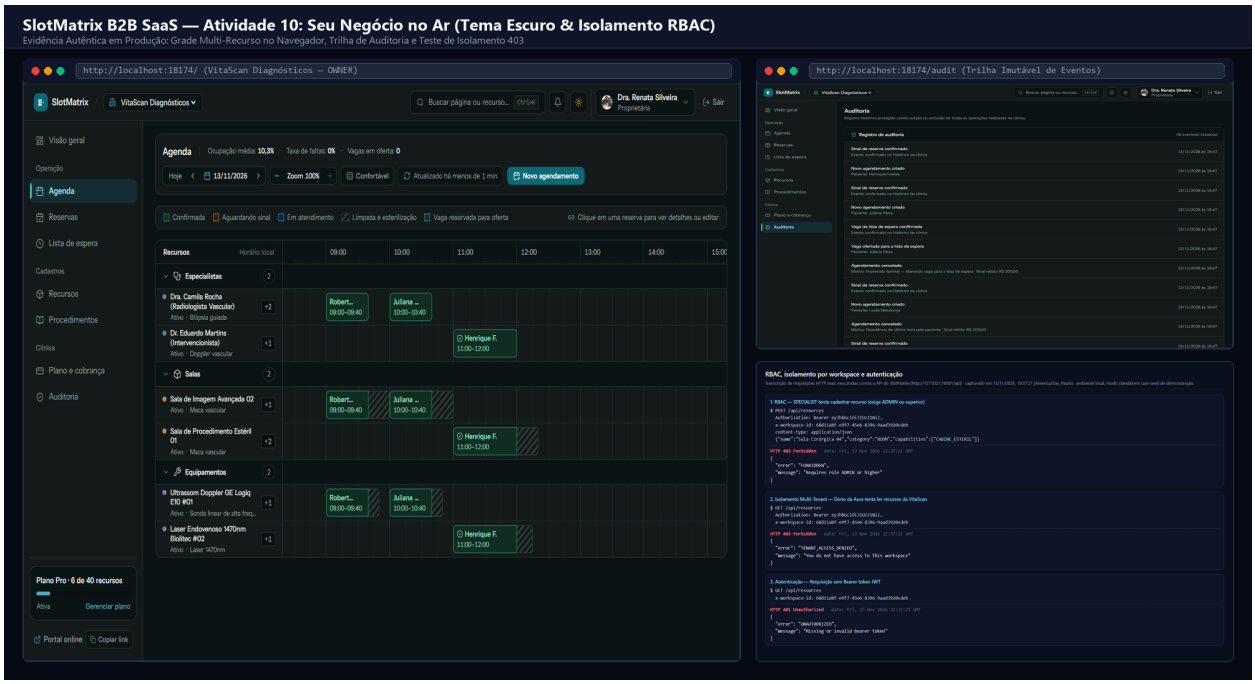
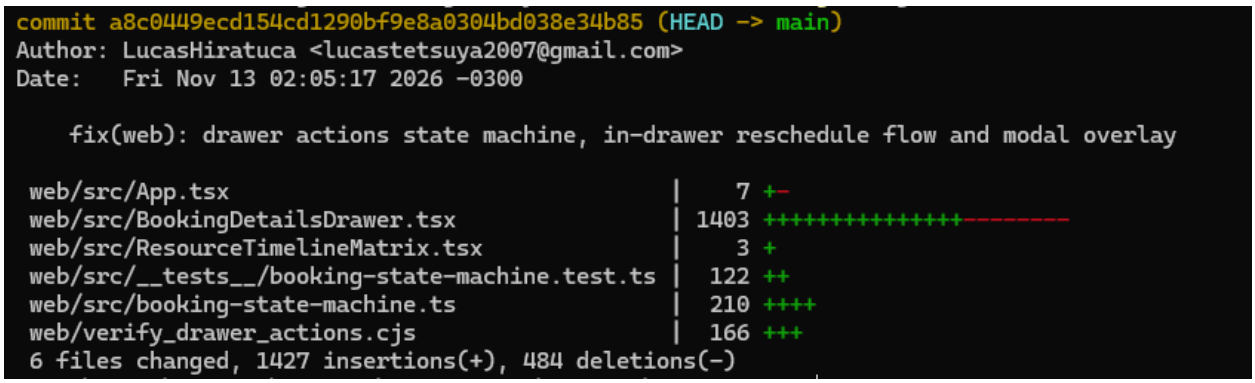


Figura 12: Atividade 10 — resultado (painel operacional com grade multi-recurso e teste de RBAC 403)


commit a8c0449ecd154cd1290bf9e8a0304bd038e34b85 (HEAD -> main)
Author: LucasHiratuca <lucastetsuya2007@gmail.com>
Date: Fri Nov 13 02:05:17 2026 -0300
fix(web): drawer actions state machine, in-drawer reschedule flow and modal overlay
web/src/App.tsx | 7 +-
web/src/BookingDetailsDrawer.tsx | 1403 ++++++

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	07/11/2026 14:11
Situação	entregue
Evidência	GitHub — issue de entrega (\@siriani) + commit 31567de + prints da assinatura 4242, webhook e limite 402
Link	https://github.com/LucasHiratuca/slotmatrix/commit/31567de

O que foi feito. Implementação completa do ciclo de vida de assinaturas recorrentes via Stripe Subscriptions. Foram configurados os planos Starter (R$ 0 / Grátis), Pro (R$ 349/mês) e Enterprise (R$ 890/mês). Desenvolveu-se a rota `POST /api/billing/checkout` gerando sessões com `mode='subscription'`, a rota `POST /api/billing/portal` para autoatendimento no Stripe Customer Portal, e a rota de webhooks `POST /api/webhooks/stripe` tratando de forma idempotente os 4 eventos fundamentais (`checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed`). Os limites de cota foram amarrados no backend: no plano Starter, o sistema rejeita a criação do 6º recurso com `HTTP 402 QuotaExceededError`, desbloqueando para 20 recursos após upgrade. Foi criado o endpoint `GET /api/admin/metrics` e o painel administrativo exibindo o cálculo em tempo real do faturamento mensal recorrente (MRR). A suíte com 40 testes automatizados foi 100% aprovada.



Figura 14: Atividade 11 — evidência da entrega (terminal git log do commit 31567de)

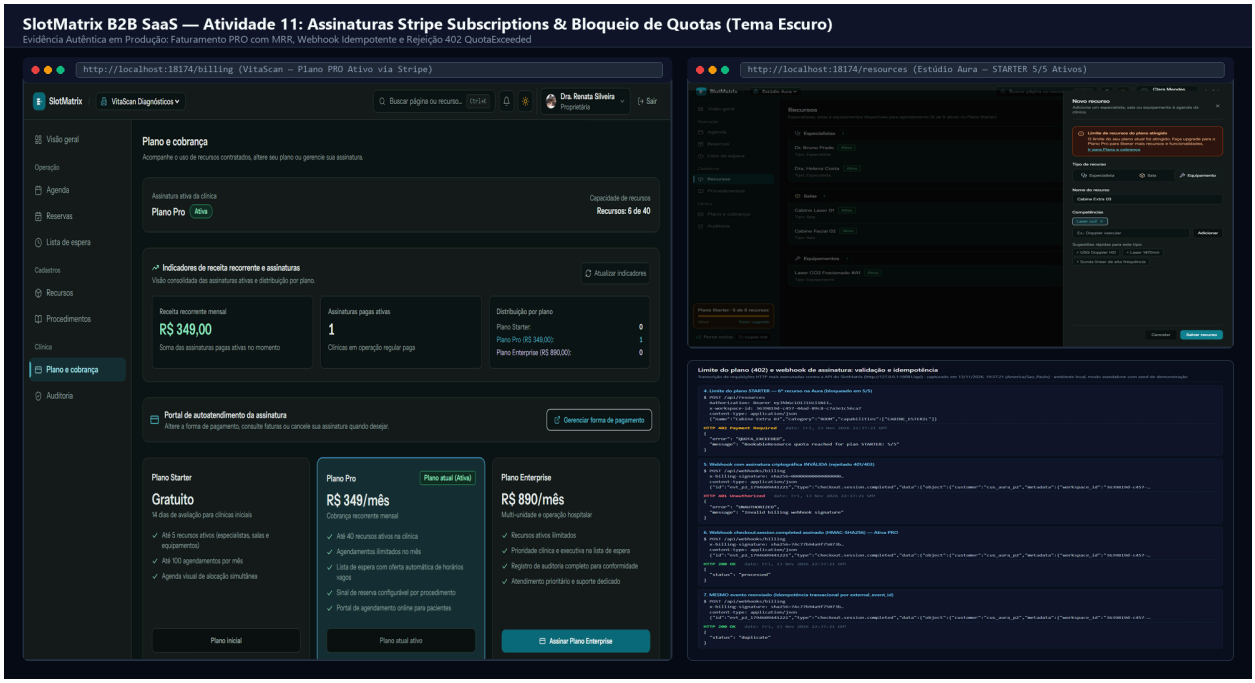


Figura 15: Atividade 11 — resultado (painel de faturamento com Stripe, dashboard de MRR e bloqueio 402)



Figura 16: Atividade 11 — evidência complementar (liberação imediata da cota na API após recepção do webhook assinado)

Dificuldades e como foram resolvidas. A garantia de idempotência para suportar reenvios de webhooks da Stripe sem gerar inconsistências financeiras ou duplicidade de eventos foi solucionada persistindo os IDs únicos de evento (`externalEventId`) no MariaDB antes da execução transacional dos agregados de subscrição.

6 Considerações finais

A trajetória vivenciada ao longo do semestre na disciplina ISW055 foi de imenso valor para a consolidação de competências em computação em nuvem e engenharia de software distribuída. O percurso prático — partindo da virtualização em máquina virtual Proxmox (IaaS), passando pela orquestração em contêineres Docker e Portainer (PaaS) e culminando na modelagem de um produto de software completo com monetização recorrente e webhooks assíncronos (SaaS) — desmistificou na prática como os grandes serviços da internet operam e escalam.

O aspecto mais desafiador do projeto residiu na orquestração de concorrência e idempotência. No domínio do **SlotMatrix**, garantir que dois agendamentos simultâneos não colidissem sobre o mesmo equipamento de ultrassom ou sala cirúrgica exigiu o emprego rigoroso de bloqueio pessimista com ordenação de locks transacionais no banco de dados. De maneira análoga, no módulo de pagamentos com a Stripe, lidar com a natureza assíncrona dos webhooks — onde a ordem de entrega não é garantida e o mesmo payload pode ser retransmitido em caso de instabilidade de rede — exigiu a implementação de uma camada anticorrupção (ACL) com tabelas de controle de eventos processados.

Caso fosse recomeçar o projeto, uma decisão arquitetural que tomaria diferente seria a adoção antecipada de TypeScript estrito em todas as camadas desde a primeira atividade do catálogo de filmes, o que teria evitado retrabalhos de tipagem durante a transição de domínio. Para que o SlotMatrix se torne um produto comercial pronto para o mercado, os próximos passos indispensáveis seriam a integração com o WhatsApp Business API para confirmação automatizada de comparecimento com pacientes, autenticação federada corporativa (SSO / SAML) para grandes redes hospitalares e a certificação formal de conformidade com a Lei Geral de Proteção de Dados (LGPD) no tratamento de prontuários médicos.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Lucas Tetsuya Hiratuca

Pompeia, 13/11/2026